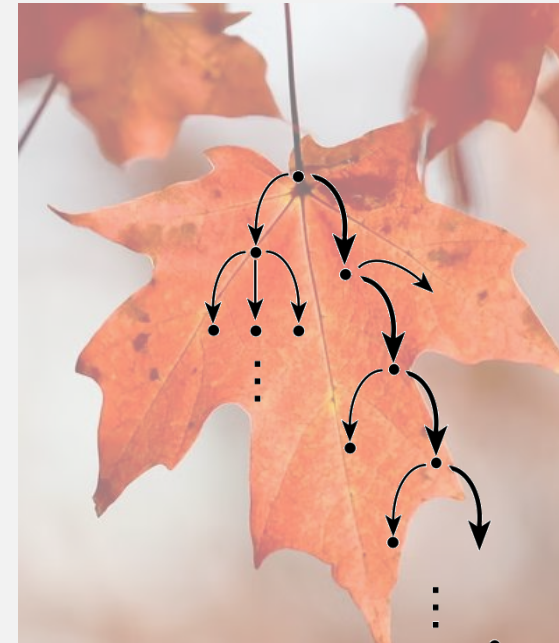


CS 420

Nondeterminism

Thursday, October 1, 2026

UMass Boston Computer Science



Announcements

- Questions / Complaints about grading?
 - GradeScope re-grade requests welcome
 - Must refer to specific deduction
 - This is the **only** way to discuss grading
 - Do not ask individual instructor or TAs
- HW 2
 - ~~Due: Tue 9/28~~
- HW 3
 - Out: Tue 9/29
 - Due: Tue 10/6 12:30pm



Last Time

Want: “Closed” Ops on Regular Langs!

- Set of Regular Languages = $\{L_1, L_2, \dots\}$
 - Closed under ...?
 - Union (“OR”) ←
 - Intersection (“AND”)
 - ...

A set is **closed** under an operation if:
the result of applying the operation to
members of the set is in the same set

For Example:

$U : \text{Language} \times \text{Language} \rightarrow \text{Language}$

Union: $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$

i.e., input set(s) = output set

We proved:

$U : \text{Regular Language} \times \text{Regular Language} \rightarrow \text{Regular Language}$

Why Care About Closed Ops on Reg Langs?

- Closed operations for regular langs preserve “regularness”
- I.e., it preserves the same computation model!
- Can “combine” smaller “regular” computations to get bigger ones:
- This semester, we will spend a lot of time looking for **operations that are closed!** (for different sets of languages)

$U : \text{Regular Language} \times \text{Regular Language} \rightarrow \text{Regular Language}$



Are Regular Languages Closed Under Union?

THEOREM

The class of regular languages is closed under the union operation.

Want to
prove this
statement



In other words,

Or this (same)
statement



IF A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

Are Regular Languages Closed Under Union?

THEOREM

The class of regular languages is closed under the union operation.

In other words,

Or this (same)
statement

IF A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

How to prove an IF-THEN statement?

Is Union Closed For Regular Langs?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language

THEN $A_1 \cup A_2$ IS-A **Regular** Language

“assume the IF part ...”

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

Justifications

7. $A_1 \cup A_2$ IS-A **Regular** Language

“... prove the THEN part”

Is Union Closed For Regular Langs?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

Justifications

DefRegLang

IF M IS-A **DFA** Machine AND M **recognizes** L
THEN L IS-A **Regular** Language

5. $\overline{M}$ ODO: construct the DFA!)
IS-A **DFA** Machine

????

6. M **recognizes** $A_1 \cup A_2$

????

7. $A_1 \cup A_2$ IS-A **Regular** Language

7. **APPLY** DefRegLang
TO mk-AND-proof Proof₅?, Proof₆?

Is Union Closed For Regular Langs?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

Problem: A_1 and A_2 are unknown languages!

Justifications

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA** Machine AND
 M_1 **recognizes** A_1

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro
TO Proof₁

But we do
know some
things ...

DefRegLang

IF M IS-A **DFA** Machine AND M **recognizes** L
THEN L IS-A **Regular** Language

5. let $M =$ (TODO: construct the DFA!)
IS-A **DFA** Machine



DefRegLangCoro

6. M **recognizes** $A_1 \cup A_2$

IF L IS-A **Regular** Language
THEN M IS-A **DFA** Machine AND M **recognizes** L

7. $A_1 \cup A_2$ IS-A **Regular** Language

7. APPLY DefRegLang
TO mk-AND-proof Proof_{5?}, Proof_{6?}

Is Union Closed For Regular Langs?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA** Machine AND
 M_1 **recognizes** A_1
4. M_2 IS-A **DFA** Machine AND
 M_2 **recognizes** A_2
5. let $M = (\text{TODO: construct the DFA!})$
IS-A **DFA** Machine
6. M **recognizes** $A_1 \cup A_2$
7. $A_1 \cup A_2$ IS-A **Regular** Language

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro
TO Proof₁
4. APPLY DefRegLangCoro
TO Proof₂
5. ????
6. ????
7. APPLY DefRegLang
TO mk-AND-proof Proof_{5?}, Proof_{6?}

Is Union Closed For Regular Langs?, i.e.,

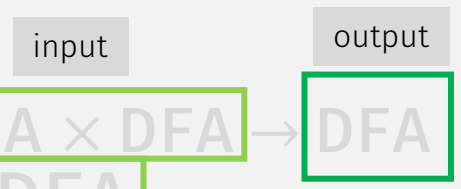
IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
 THEN $A_1 \cup A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements	Justifications
1. A_1 IS-A Regular Language	1. FIRST ASSUMPTION
2. A_2 IS-A Regular Language	2. SECOND ASSUMPTION
3. M_1 IS-A DFA Machine AND M_1 recognizes A_1	3. APPLY DefRegLangCoro TO Proof ₁
4. M_2 IS-A DFA Machine AND M_2 recognizes A_2	4. APPLY DefRegLangCoro TO Proof ₂
5. let $M = \text{UNION}_{\text{DFA}}(M_1, M_2)$ IS-A DFA Machine	5. APPLY UNION _{DFA} : DFA × DFA → DFA TO $(M_1, M_2) : \text{DFA} \times \text{DFA}$
6. $A_1 \cup A_2$ IS-A Regular Language	6. ????
7. $A_1 \cup A_2$ IS-A Regular Language	7. APPLY DefRegLang TO mk-AND-proof Proof _{5?} , Proof _{6?}

Construct M using M_1, M_2 parts

Allowed to use let to define **DFA** in an IS-A stmt



Are Regular Languages Closed Under Union?

Proof (continuation)

Define function $\text{UNION}_{\text{DFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{DFA}$

$\text{UNION}_{\text{DFA}}(M_1, M_2) = M$, where

- Input: ~~$M_1 = \text{mk-DFA}(Q_1, \Sigma, \delta_1, q_1, F_1)$~~
 ~~$M_2 = \text{mk-DFA}(Q_2, \Sigma, \delta_2, q_2, F_2)$~~
- Output: $M = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ using M_1, M_2 parts (that accepts if either accepts)
where:

(construction)
(same as before)

mk-DFA : $\text{Setof}\langle A \rangle \times \text{Setof}\langle C \rangle \times (A \times C \rightarrow A) \times A \times \text{Setof}\langle A \rangle \rightarrow \text{DFA}$

(deconstruction)
(i.e., “getter”)

get-DFA-parts : $\text{DFA} \rightarrow \text{Setof}\langle A \rangle \times \text{Setof}\langle C \rangle \times (A \times C \rightarrow A) \times A \times \text{Setof}\langle A \rangle$

NOTE:

Some students were confused with
using the same notation for

- input deconstruction (i.e., “getters”) and
- output construction ...

... so I’m changing it

Are Regular Languages Closed Under Union?

Proof (continuation)

Define function $\text{UNION}_{\text{DFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{DFA}$

$\text{UNION}_{\text{DFA}}(M_1, M_2) = M$, where

(“pattern match”)

let $(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-DFA-parts } M_1$

let $(Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-DFA-parts } M_2$

Want: M that can simultaneously “be in” both an M_1 and M_2 state

Output: $M = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ using M_1, M_2 parts (that accepts if either accepts)

where:

• states of M :

$Q = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ and } r_2 \in Q_2\} = Q_1 \times Q_2$

This set is the *Cartesian product* of sets Q_1 and Q_2

A *finite automaton* is a $\text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set called the *states*,
2. Σ is a finite set called the *alphabet*,
3. $\delta: Q \times \Sigma \rightarrow Q$ is the *transition function*,¹
4. $q_0 \in Q$ is the *start state*, and
5. $F \subseteq Q$ is the *set of accept states*.

A state of M is a pair:

- first part: state of M_1
- second part: state of M_2

states of M : all possible pair combinations of states of M_1 and M_2

Are Regular Languages Closed Under Union?

Proof (continuation)

Define function $\text{UNION}_{\text{DFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{DFA}$

$\text{UNION}_{\text{DFA}}(M_1, M_2) = M$, where

let $(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-DFA-parts } M_1$

let $(Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-DFA-parts } M_2$

Output: $M = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ using M_1, M_2 parts (that accepts if either accepts)
where:

• states of M :

$Q = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ and } r_2 \in Q_2\} = Q_1 \times Q_2$

This set is the *Cartesian product* of sets Q_1 and Q_2

A *finite automaton* is a $\text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set called the *states*,
2. Σ is a finite set called the *alphabet*,
3. $\delta: Q \times \Sigma \rightarrow Q$ is the *transition function*,
4. $q_0 \in Q$ is the *start state*, and
5. $F \subseteq Q$ is the *set of accept states*.

$\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$

A step in M is both:
- a step in M_1 , and
- a step in M_2

Are Regular Languages Closed Under Union?

Proof (continuation)

Define function $\text{UNION}_{\text{DFA}} : \mathbf{DFA} \times \mathbf{DFA} \rightarrow \mathbf{DFA}$

$\text{UNION}_{\text{DFA}} (M_1, M_2) = M$, where

let $(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-DFA-parts } M_1$

let $(Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-DFA-parts } M_2$

Output: $M = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ using M_1, M_2 parts (that accepts if either accepts)
where:

- states of M : $Q = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ and } r_2 \in Q_2\} = Q_1 \times Q_2$
This set is the *Cartesian product* of sets Q_1 and Q_2
- M transition fn: $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$
- M start state: $q_0 = (q_1, q_2)$

Start state of M is both
start states of M_1 and M_2

Are Regular Languages Closed Under Union?

Proof (continuation)

Define function $\text{UNION}_{\text{DFA}} : \mathbf{DFA} \times \mathbf{DFA} \rightarrow \mathbf{DFA}$

$\text{UNION}_{\text{DFA}}(M_1, M_2) = M$, where

let $(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-DFA-parts } M_1$

let $(Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-DFA-parts } M_2$

Output: $M = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ using M_1, M_2 parts (that accepts if either accepts)
where:

- states of M : $Q = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ and } r_2 \in Q_2\} = Q_1 \times Q_2$
This set is the *Cartesian product* of sets Q_1 and Q_2

- M transition fn: $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$

- M start state: $q_0 = (q_1, q_2)$

- M accept states: $F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ or } r_2 \in F_2\}$

Remember:
Accept states must
be subset of Q

Accept if either M_1 or M_2 accept

Is Union Closed For Regular Langs?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA** Machine AND
 M_1 **recognizes** A_1
4. M_2 IS-A **DFA** Machine AND
 M_2 **recognizes** A_2
5. let $M = \text{UNION}_{\text{DFA}}(M_1, M_2)$
IS-A DFA Machine
6. M **recognizes** $A_1 \cup A_2$
7. $A_1 \cup A_2$ IS-A **Regular** Language

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro
TO Proof₁
4. APPLY DefRegLangCoro
TO Proof₂
5. APPLY $\text{UNION}_{\text{DFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{DFA}$
TO $(M_1, M_2) : \text{DFA} \times \text{DFA}$
6. ????
7. APPLY DefRegLang
TO mk-AND-proof Proof₅, Proof₆

Is Union Closed For Regular Langs?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA** Machine AND
 M_1 **recognizes** A_1
4. M_2 IS-A **DFA** Machine AND
 M_2 **recognizes** A_2
5. let $M = \text{UNION}_{\text{DFA}}(M_1, M_2)$
IS-A **DFA** Machine
6. M **recognizes** $A_1 \cup A_2$
7. $A_1 \cup A_2$ IS-A **Regular** Language

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro
TO Proof₁
4. APPLY DefRegLangCoro
TO Proof₂
5. APPLY $\text{UNION}_{\text{DFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{DFA}$
TO $(M_1, M_2) : \text{DFA} \times \text{DFA}$
6. BY (TODO: create Example Table!) **COMPARING** $M, A_1 \cup A_2$
7. APPLY DefRegLang
TO mk-AND-proof Proof₅, Proof₆

“Prove” that DFA recognizes a language

let $s_1 \in A_1, s_2 \in A_2$

let $s_3 \notin A_1, s_4 \notin A_2$

Be careful when choosing examples!

In this class, a table like this is sufficient to “prove” that a DFA recognizes a language

let $s =$	$s \in A_1 \cup A_2?$	run M on $s?$
s_1	Yes	
s_2	Yes	
s_3	???	
s_4	???	

From:

Don't know A_1 and A_2 exactly ...

... but we know ...

... they are **sets of strings!**

let $(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-DFA-parts } M_1$

recognizes A_1

let $(Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-DFA-parts } M_2$

recognizes A_2

Constructed:

$M = \text{UNION}_{\text{DFA}}(M_1, M_2) = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ that recognizes $A_1 \cup A_2?$

“Prove” that DFA recognizes a language

let $s_1 \in A_1, s_2 \in A_2$

~~let $s_3 \notin A_1, s_4 \notin A_2$~~

let $s_5 \notin A_1$ and $\notin A_2$

let $s =$	$s \in A_1 \cup A_2?$	run M on $s?$
s_1	Yes	
s_2	Yes	
s_3	???	
s_4	???	
s_5	No	

From:

let $(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-DFA-parts } M_1$ recognizes A_1

let $(Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-DFA-parts } M_2$ recognizes A_2

Constructed:

$M = \text{UNION}_{\text{DFA}}(M_1, M_2) = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ that recognizes $A_1 \cup A_2?$

Define function $\text{UNION}_{\text{DFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{DFA}$

$\text{UNION}_{\text{DFA}} (M_1, M_2) = M$, where

let $(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-DFA-parts } M_1$

let $(Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-DFA-parts } M_2$

$M = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$

where:

- **states of M :** $Q = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ and } r_2 \in Q_2\} = Q_1 \times Q_2$
This set is the *Cartesian product* of sets Q_1 and Q_2
- M transition fn: $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$
- M start state: $q_0 = (q_1, q_2)$
- M accept states: $F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ or } r_2 \in F_2\}$.

Accept if either M_1 or M_2 accept

“Prove” that DFA recognizes a language

let $s_1 \in A_1, s_2 \in A_2$

let $s_5 \notin A_1, \notin A_2$

(this column needed when machine is not concrete, i.e., can't directly run machine to check if string is accepted)

let $s =$	$s \in A_1 \cup A_2?$	run M on $s?$	run M on s Justification
s_1	Yes	Accept ??	??
s_2	Yes	Accept ??	??
s_3	???		
s_4	???		
s_5	No	Reject ??	??

From:

let $(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-DFA-parts } M_1$ recognizes A_1

let $(Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-DFA-parts } M_2$ recognizes A_2

Accept if either M_1 or M_2 accept

where:

$$F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ or } r_2 \in F_2\}.$$

Constructed:

$M = \text{UNION}_{\text{DFA}}(M_1, M_2) = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ that recognizes $A_1 \cup A_2$

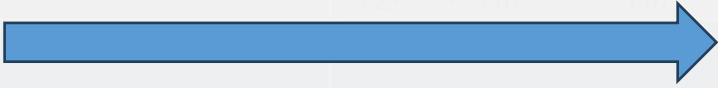
“Prove” that DFA recognizes a language

let $s_1 \in A_1, s_2 \in A_2$

let $s_5 \notin A_1, \notin A_2$

(these extra columns are allowed / required when machine is not concrete, and constructed from smaller machines)

(this column needed when machine is not concrete, i.e., can't directly run machine to check if string is accepted)

let $s =$	$s \in A_1 \cup A_2?$	run M_1 on $s =$	run M_2 on $s =$	run M on $s?$	run M on s Justification
s_1	Yes	$q_1, \dots, q_n$ where $q_n \in F_1$		Accept ??	$(q_1, q_2), \dots, (q_n, _)??$ where $(q_n, _) \in F$
s_2	Yes		$q_1, \dots, q_m$ where $q_m \in F_2$	Accept ??	$(q_1, q_2), \dots, (_, q_m)??$ where $(_, q_m) \in F$
s_3	???			???	
s_4	???			???	
s_5	No	$q_1, \dots, q_n$ where $q_n \notin F_1$	$q_1, \dots, q_m$ where $q_m \notin F_2$	Reject ??	$(q_1, q_2), \dots, (q_n, q_m)??$ where $(q_n, q_m) \notin F$

From:

let $(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-DFA-parts } M_1$ recognizes A_1

let $(Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-DFA-parts } M_2$ recognizes A_2

where:

Constructed:

$F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ or } r_2 \in F_2\}$

$M = \text{UNION}_{\text{DFA}}(M_1, M_2) = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ that recognizes $A_1 \cup A_2?$

let $\text{ExTable}_1 =$

“Prove” that DFA recognizes a language

let $s_1 \in A_1, s_2 \in A_2$

let $s_5 \notin A_1, \notin A_2$

(these extra columns are allowed / required when machine is not concrete, and constructed from smaller machines)

(this column needed when machine is not concrete, i.e., can't directly run machine to check if string is accepted)

let $s =$	$s \in A_1 \cup A_2?$	run M_1 on $s =$	run M_2 on $s =$	run M on $s?$	run M on s Justification
s_1	Yes	$q_1, \dots, q_n$ where $q_n \in F_1$		Accept ??	$(q_1, q_2), \dots, (q_n, _)$?? where $(q_n, _) \in F$
s_2	Yes	A run of a machine = sequence of states	$q_2, \dots, q_m$ where $q_m \in F_2$	Accept ??	$(q_1, q_2), \dots, (_, q_m)$?? where $(_, q_m) \in F$
s_3	???	An accepting run ends in accept state!	A rejecting run <u>does not</u> end in accept state!	???	
s_4	???			???	
s_5	No	$q_1, \dots, q_r$ where $q_r \notin F_1$	$q_2, \dots, q_s$ where $q_s \notin F_2$	Reject ??	$(q_1, q_2), \dots, (q_r, q_s)$?? where $(q_r, q_s) \notin F$

From:

let $(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-DFA-parts } M_1$ recognizes A_1

let $(Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-DFA-parts } M_2$ recognizes A_2

Reject if neither M_1 or M_2 accept

Accept if either M_1 or M_2 accept

where:

Constructed:

$$F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ or } r_2 \in F_2\}$$

$M = \text{UNION}_{\text{DFA}}(M_1, M_2) = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ that recognizes $A_1 \cup A_2?$

Previously

Is Union Closed For Regular Langs?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

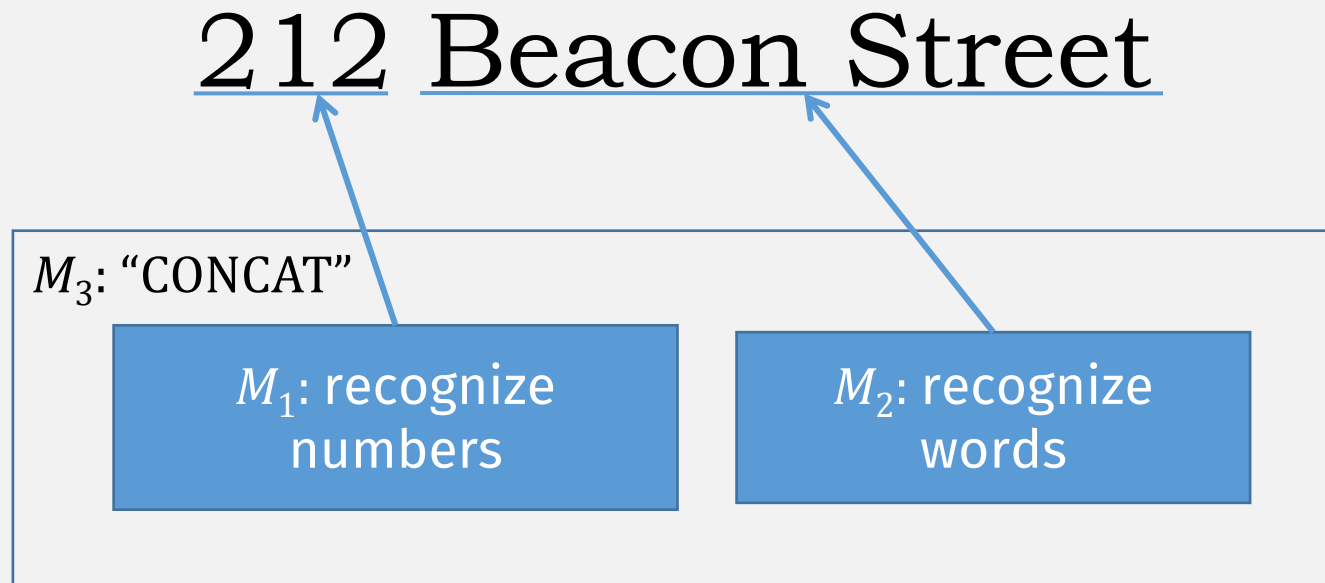
1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA** Machine AND
 M_1 **recognizes** A_1
4. M_2 IS-A **DFA** Machine AND
 M_2 **recognizes** A_2
5. let $M = \text{UNION}_{\text{DFA}}(M_1, M_2)$
IS-A **DFA** Machine
6. M **recognizes** $A_1 \cup A_2$
7. $A_1 \cup A_2$ IS-A **Regular** Language

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro
TO Proof₁
4. APPLY DefRegLangCoro
TO Proof₂
5. APPLY $\text{UNION}_{\text{DFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{DFA}$
TO $M_1 : \text{DFA}, M_2 : \text{DFA}$
6. BY (TODO: create Example Table!) **COMPARING** $M, A_1 \cup A_2$
7. APPLY DefRegLang
TO mk-AND-proof Proof₅, Proof₆

Another (common string) operation: Concatenation

Example: Recognizing street addresses



Concatenation: $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$

(infix function notation)

Concatenation of Languages

Let the alphabet Σ be the standard 26 letters $\{a, b, \dots, z\}$.

If $A = \{\text{fort}, \text{south}\}$ $B = \{\text{point}, \text{boston}\}$

$$A \circ B = \{\text{fortpoint}, \text{fortboston}, \text{southpoint}, \text{southboston}\}$$

Regular Languages Closed Under Concat?

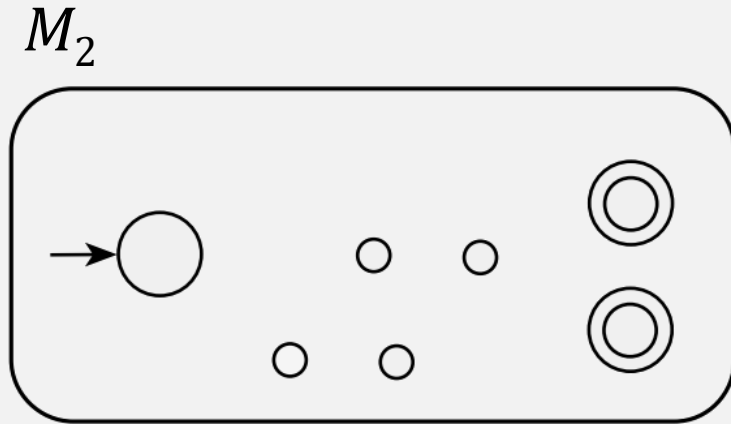
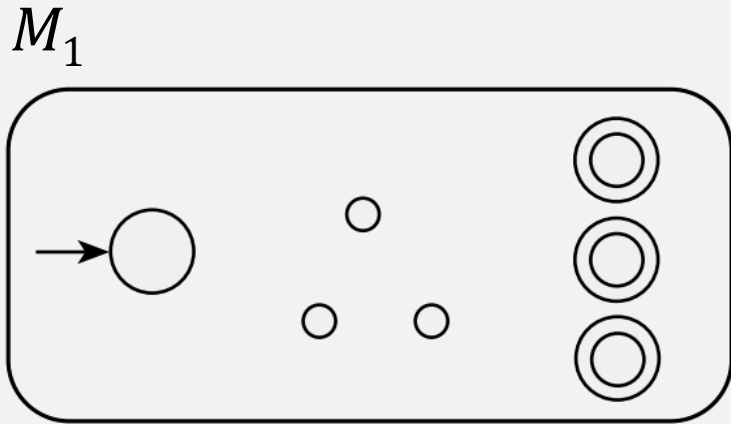
THEOREM

The class of regular languages is closed under the concatenation operation.

In other words, IF A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language
THEN $A_1 \circ A_2$ IS-A **Regular** Language

- Construct a new machine M recognizing $A_1 \circ A_2$? (like union)
 - Using DFA M_1 (which recognizes A_1),
 - and DFA M_2 (which recognizes A_2)

Concatentation

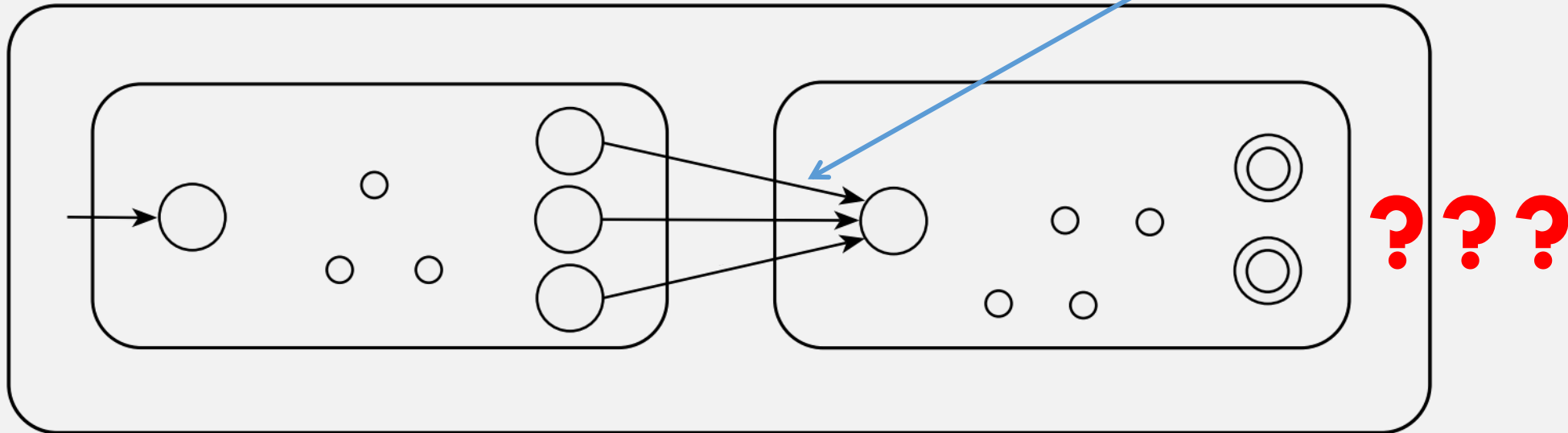


PROBLEM:
Can only
read input
once, can't
backtrack

Let M_1 recognize A_1 , and M_2 recognize A_2 .

Want: Construction of M to recognize $A_1 \circ A_2$

Need to switch
machines at some
point, but when?



Concatenation: $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$

Overlapping Concatenation Example

- Let M_1 ... recognize language $A = \{ \text{jen}, \text{jens} \}$
- and M_2 ... recognize language $B = \{ \text{smith} \}$
- Want: Construct M to recognize $A \circ B = \{ \boxed{\text{jen}}\text{smith}, \boxed{\text{jens}}\text{smith} \}$
- If M sees **jen** ...
- M must decide to either:

Concatenation: $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$

Overlapping Concatenation Example

- Let M_1 ... recognize language $A = \{ \text{jen}, \text{jens} \}$
- and M_2 ... recognize language $B = \{ \text{smith} \}$
- Want: Construct M to recognize $A \circ B = \{ \text{jensmith}, \text{jenssmith} \}$
- If M sees **jen** ...
- M must decide to either:
 - stay in M_1 (correct, if full input is **jen**~~s~~**smith**)

Concatenation: $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$

Overlapping Concatenation Example

- Let M_1 ... recognize language $A = \{ \text{jen}, \text{jens} \}$
- and M_2 ... recognize language $B = \{ \text{smith} \}$
- Want: Construct M to recognize $A \circ B = \{ \text{jensmith}, \text{jenssmith} \}$

- If M sees jen ...

- M must decide to either:

- stay in M_1 (correct, if full input is jenssmith)
 - or switch to M_2 (correct, if full input is jenssmith)

- But to recognize $A \circ B$, it needs to handle both cases!!

- Without backtracking

A DFA can't do this!

(seemingly)

Regular Languages Closed Under Concat?

FALSE?

THEOREM

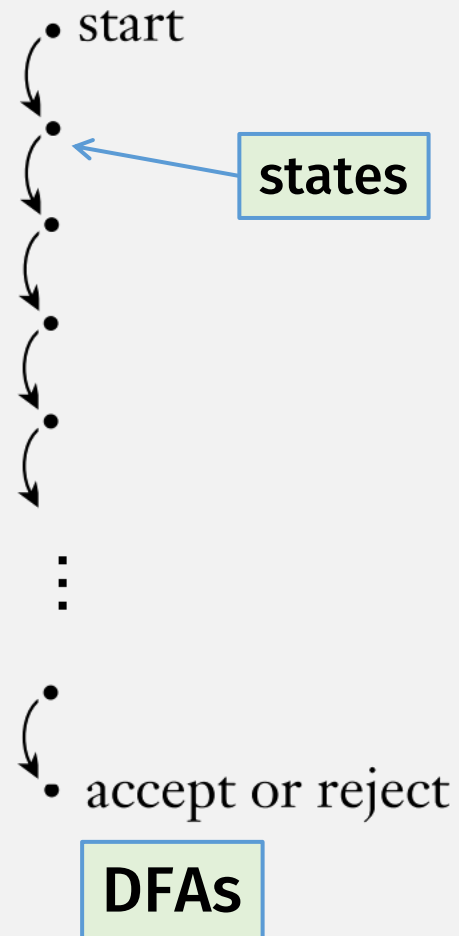
The class of regular languages is closed under the concatenation operation.

- Cannot combine A_1 and A_2 's machine because:
 - Need to switch from A_1 to A_2 at some point ...
 - ... but we don't know when! (we can only read input once)
- This requires a new kind of machine!
- But does this mean concatenation is not closed for regular langs?

Nondeterminism

Deterministic vs Nondeterministic

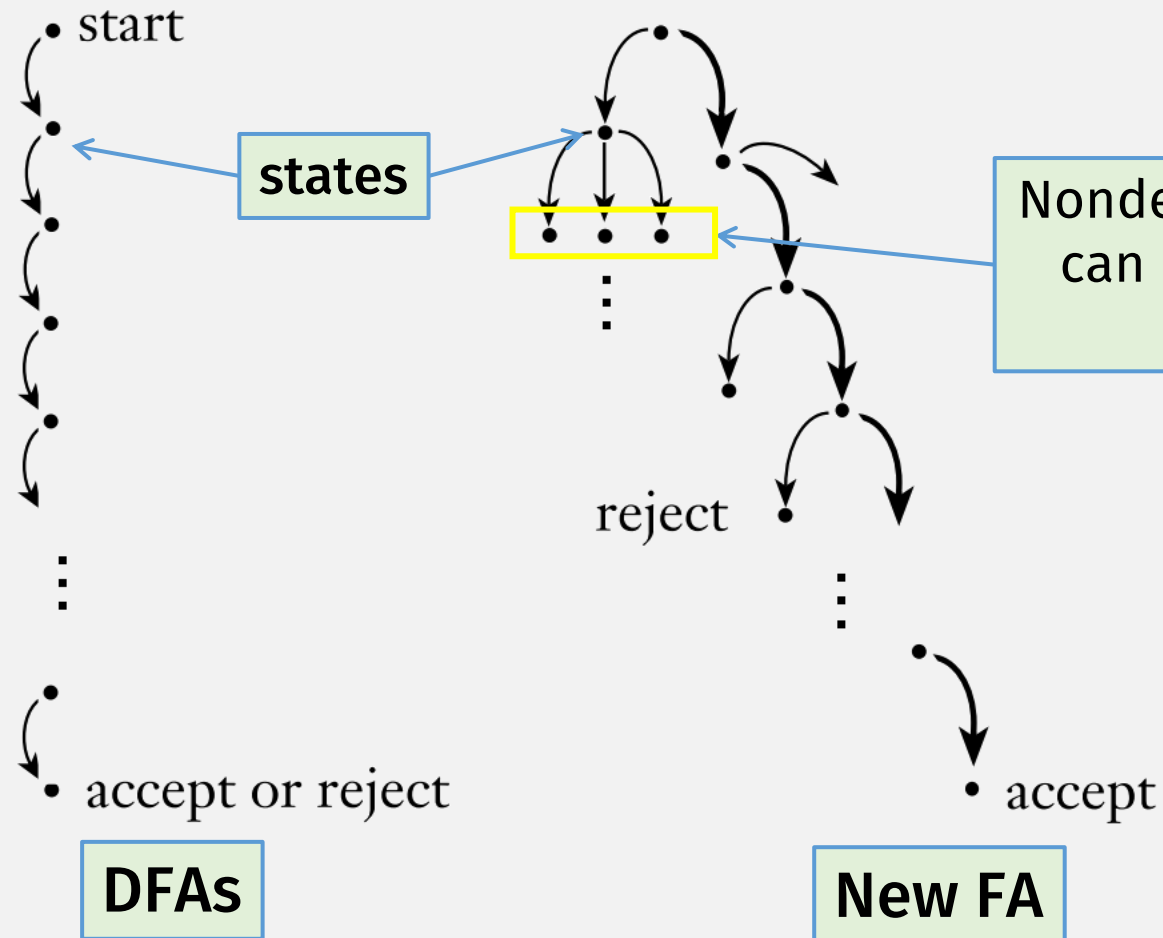
Deterministic
computation



Deterministic vs Nondeterministic

Deterministic
computation

Nondeterministic
computation



DFA: The Formal Definition

DEFINITION

deterministic

A *finite automaton* is a mk-DFA $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set called the *states*,
2. Σ is a finite set called the *alphabet*,
3. $\delta: Q \times \Sigma \longrightarrow Q$ is the *transition function*,
4. $q_0 \in Q$ is the *start state*, and
5. $F \subseteq Q$ is the *set of accept states*.

Deterministic Finite Automata (DFA)

Nondeterministic Finite Automata (NFA)

DefNFA

DEFINITION

A *nondeterministic finite automaton* is a **mk-NFA** $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set of states,
2. Σ is a finite alphabet,
3. $\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$ is the transition function,
4. $q_0 \in Q$ is the start state, and
5. $F \subseteq Q$ is the set of accept states.

Difference

Power set, i.e. a transition results in set of states

Compare with DFA:

A *finite automaton* is a **mk-DFA** $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set called the *states*,
2. Σ is a finite set called the *alphabet*,
3. $\delta: Q \times \Sigma \longrightarrow Q$ is the *transition function*,
4. $q_0 \in Q$ is the *start state*, and
5. $F \subseteq Q$ is the *set of accept states*.

Power Sets

- A **power set** is the set of all subsets of a set
- Example: $S = \{a, b, c\}$
- Power set of $S =$
 - $\{ \{ \}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\} \}$
 - Note: includes the empty set!

Nondeterministic Finite Automata (NFA)

DEFINITION

A *nondeterministic finite automaton*

is a **mk-NFA** $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set of states,
2. Σ is a finite alphabet,
3. $\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$ is the transition function,
4. $q_0 \in Q$ is the start state, and
5. $F \subseteq Q$ is the set of accept states.

Transition label can be “empty”,
i.e., machine can transition
without reading input

$$\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$$

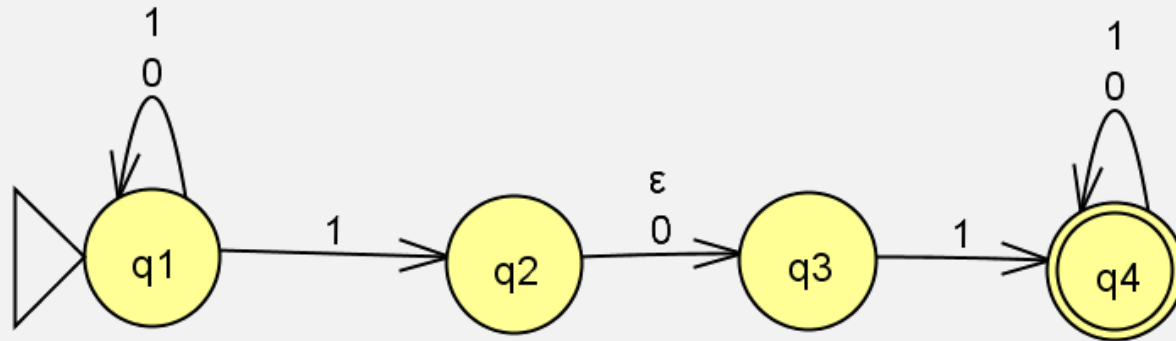
CAREFUL:

ϵ symbol is reused here, as a transition label
(ie, an argument to δ)

- **It's not the empty string!**
- And it's (still) not a character in the alphabet Σ !

NFA Example

- Come up with a formal description of the following NFA:



DEFINITION

A *nondeterministic finite automaton*

is a **mk-NFA** $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set of states,
2. Σ is a finite alphabet,
3. $\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$ is the transition function,
4. $q_0 \in Q$ is the start state, and
5. $F \subseteq Q$ is the set of accept states.

let $N_1 = \text{mk-NFA}(Q, \Sigma, \delta, q_1, F)$, where

$$Q = \{q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{0,1\}$$

$$\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$$

Empty transition
(no input read)

Result of transition
is a set

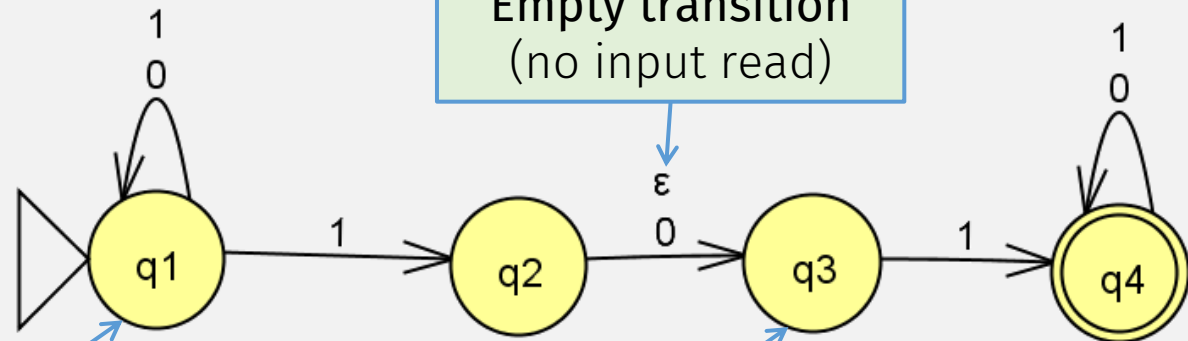
	0	1	ϵ
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

$$F = \{q_4\}$$

Empty transition
(no input read)

Multiple 1 transitions

No 0 transition



In-class Exercise

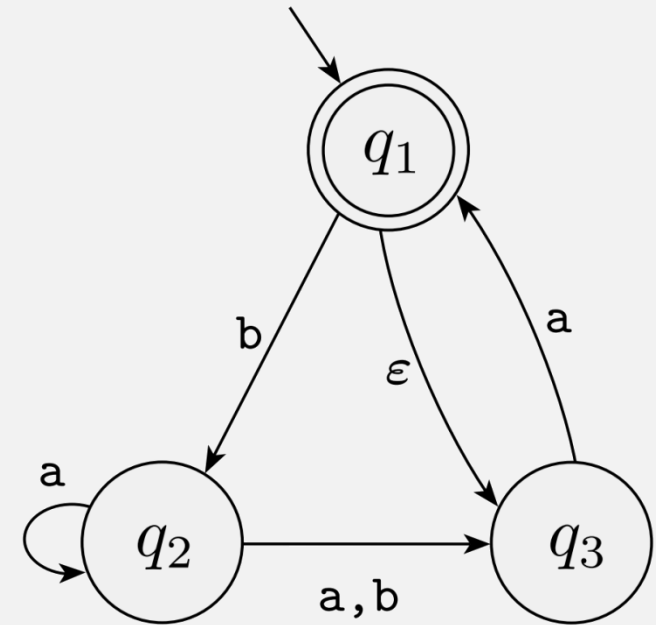
- Come up with a formal description for the following NFA
 $\Sigma = \{ \mathbf{a}, \mathbf{b} \}$

DEFINITION

A *nondeterministic finite automaton*

is a **mk-NFA** $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set of states,
2. Σ is a finite alphabet,
3. $\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$ is the transition function,
4. $q_0 \in Q$ is the start state, and
5. $F \subseteq Q$ is the set of accept states.



In-class Exercise Solution

let $N = \text{mk-NFA}(Q, \Sigma, \delta, q_0, F)$, where

$$Q = \{q_1, q_2, q_3\}$$

$$\Sigma = \{a, b\}$$

$\delta \dots \longrightarrow$

$$q_0 = q_1$$

$$F = \{q_1\}$$

$$\delta(q_1, a) = \{\}$$

$$\delta(q_1, b) = \{q_2\}$$

$$\delta(q_1, \varepsilon) = \{q_3\}$$

$$\delta(q_2, a) = \{q_2, q_3\}$$

$$\delta(q_2, b) = \{q_3\}$$

$$\delta(q_2, \varepsilon) = \{\}$$

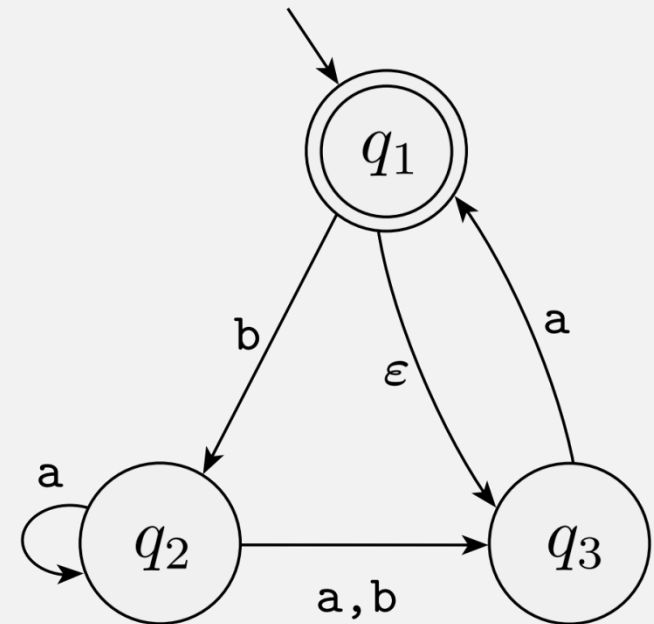
$$\delta(q_3, a) = \{q_1\}$$

$$\delta(q_3, b) = \{\}$$

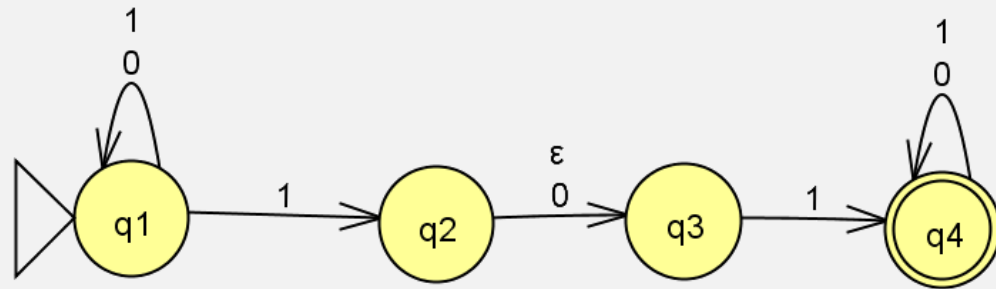
$$\delta(q_3, \varepsilon) = \{\}$$

Differences with DFA?

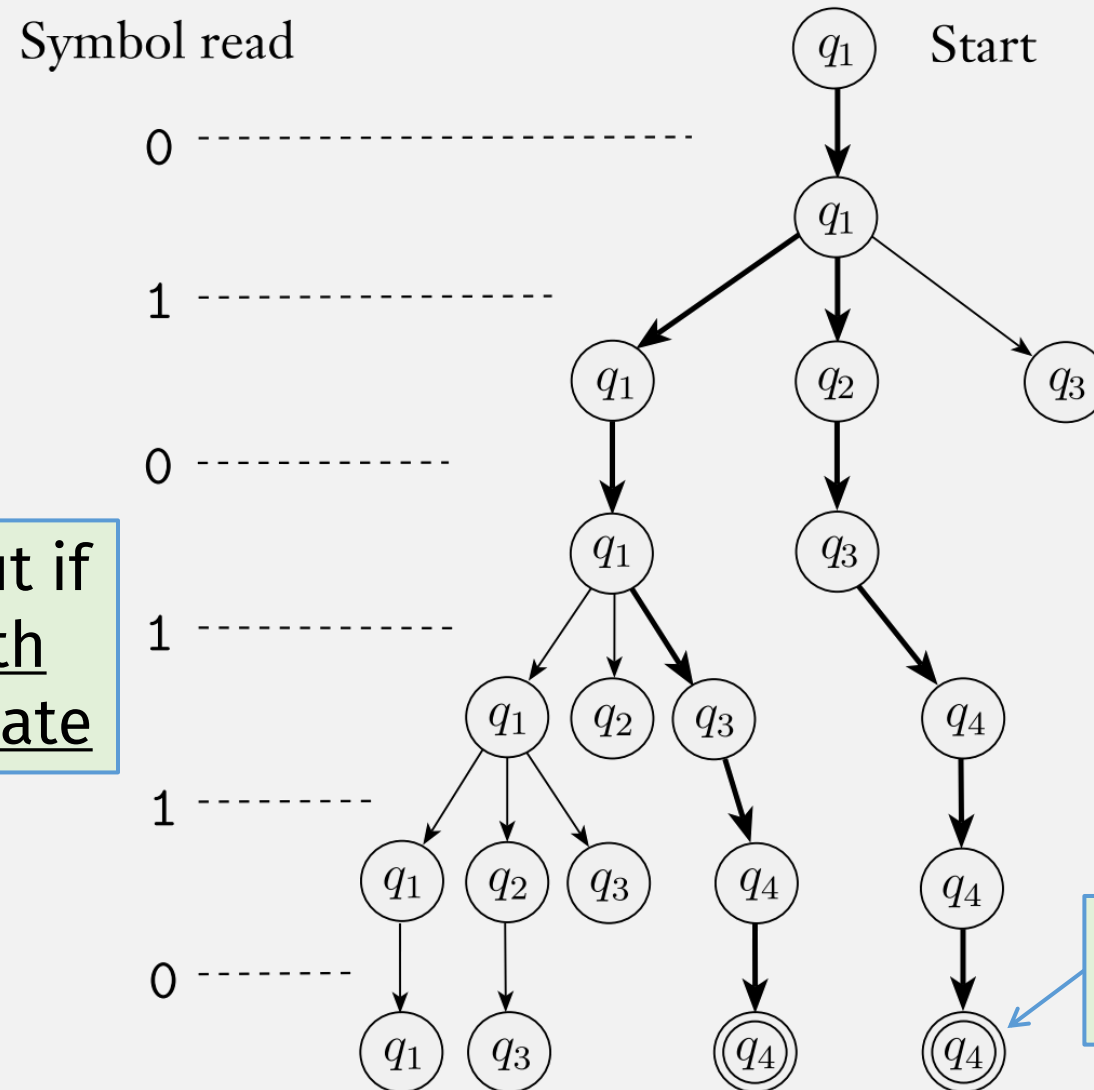
- δ output is a set
- state doesn't need transition for every alphabet symbol
- state can have multiple transitions for one symbol
- can have "empty" transitions (δ output is empty set)



NFA Computation (JFLAP demo): **010110**



NFA Computation Sequence



NFA **accepts** input if
at least one path
ends in accept state

Each step can
branch into
multiple states at
the same time!

So this is an **accepting computation**

DFA Computation Rules

Informally

Given

- A **DFA** (~ a “Program”)
- and **Input** = **string of chars**, e.g. “1101”

A DFA computation (~ “Program run”):

- Start in *start state*
- Repeat:
 - Read 1 char from **Input**, and
 - Change state according to *transition rules*

Result of computation:

- Accept if last state is *Accept state*
- Reject otherwise

Formally (i.e., mathematically)

- $M = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F) \dots$
- $w = w_1 w_2 \cdots w_n$

A DFA **computation** is a sequence of states:

- specified by $\hat{\delta}(q_0, w)$ where:

- M **accepts** w if $\hat{\delta}(q_0, w) \in F$
- M **rejects** otherwise

DFA Computation Rules

Informally

Given

- A **DFA** (~ a “Program”)
- and **Input** = string of chars, e.g. “1101”

A **DFA computation** (~ “Program run”):

- Start in *start state*
- Repeat:
 - Read 1 char from **Input**, and
 - Change state according to *transition rules*

Result of computation:

- Accept if last state is *Accept state*
- Reject otherwise

Formally (i.e., mathematically)

- $M = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F) \dots$
- $w = w_1 w_2 \cdots w_n$

A **DFA computation** is a sequence of states:

- specified by $\hat{\delta}(q_0, w)$ where:

- M **accepts** w if $\hat{\delta}(q_0, w) \in F$
- M **rejects** otherwise

Ignoring ϵ transitions, for now!

NFA Computation Rules

Informally

Given

- An **NFA** (~ a “Program”)
- and **Input** = string of chars, e.g. “1101”

An **NFA computation** (~ “Program run”):

- Start in *start state*
- Repeat:
 - Read 1 char from **Input**, and

For each “current” state, according to *transition rules*
go to next states

... then combine all “next states”

Result of computation:

- Accept if last **set of states has accept state**
- Reject otherwise

Formally (i.e., mathematically)

- $N = \text{mk-NFA}(Q, \Sigma, \delta, q_0, F) \dots$
- $w = w_1 w_2 \cdots w_n$

An **NFA computation** is a ...

- specified by $\hat{\delta}(q_0, w)$ where:

- M **accepts** w if ...
- M **rejects** ...

Ignoring ϵ transitions, for now!

NFA Computation Rules

Informally

Given

- An **NFA** (~ a “Program”)
- and **Input** = string of chars, e.g. “1101”

An **NFA computation** (~ “Program run”):

- Start in *start state*
- Repeat:
 - Read 1 char from **Input**, and

For each “current” state, according to *transition rules*
go to next states

... then combine all “next states”

Result of computation:

- Accept if last **set of states has accept state**
- Reject otherwise

Formally (i.e., mathematically)

- $N = \text{mk-NFA}(Q, \Sigma, \delta, q_0, F) \dots$
- $w = w_1 w_2 \cdots w_n$

An **NFA computation** is a sequence of: sets of states

- specified by $\hat{\delta}(q_0, w)$ where:

???

- M **accepts** w if ...
- M **rejects** ...

DFA Multi-Step Transition Function

$$\hat{\delta} : Q \times \Sigma^* \rightarrow Q$$

- Domain (inputs):
 - state $q \in Q$ (doesn't have to be start state)
 - string $w = w_1 w_2 \cdots w_n$ where $w_i \in \Sigma$
- Range (output):
 - state $q \in Q$ (doesn't have to be an accept state)

Recursive Input Data
needs
Recursive Function

Base case

Base case

$$\hat{\delta}(q, \epsilon) =$$

A **String** is either:

- the **empty string** (ϵ), or
- xa (non-empty string) where
 - x is a **string**
 - a is a "char" in Σ

DFA Multi-Step Transition Function

$$\hat{\delta} : Q \times \Sigma^* \rightarrow Q$$

- Domain (inputs):
 - state $q \in Q$ (doesn't have to be start state)
 - string $w = w_1 w_2 \cdots w_n$ where $w_i \in \Sigma$
- Range (output):
 - state $q \in Q$ (doesn't have to be an accept state)

(Defined recursively)

Base case $\hat{\delta}(q, \varepsilon) = q$

Recursive Case

$$\hat{\delta}(q, w'w_n) = \delta(\hat{\delta}(q, w'), w_n)$$

where $w' = w_1 \cdots w_{n-1}$

Recursive Input Data
needs
Recursive Function

A **String** is either:

- the **empty string** (ε), or
- xa (non-empty string) where

- x is a **string**
- a is a "char" in Σ

Recursion
on string

string

char

Recursive case

"smaller" argument

"second to last" state

DFA Multi-Step Transition Function

$$\hat{\delta}: Q \times \Sigma^* \rightarrow Q$$

- Domain (inputs):
 - state $q \in Q$ (doesn't have to be start state)
 - string $w = w_1 w_2 \cdots w_n$ where $w_i \in \Sigma$
- Range (output):
 - state $q \in Q$ (doesn't have to be an accept state)

(Defined recursively)

Base case $\hat{\delta}(q, \varepsilon) = q$

Recursive Case

$$\hat{\delta}(q, w'w_n) = \delta(\hat{\delta}(q, w'), w_n)$$

where $w' = w_1 \cdots w_{n-1}$

Single step from “second to last” state and last char gets to last state

Recursive Input Data
needs
Recursive Function

A **String** is either:

- the **empty string** (ε), or
- xa (non-empty string) where
 - x is a **string**
 - a is a “char” in Σ

$\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$ is the transition function

NFA Multi-Step Transition Function

$$\hat{\delta}: Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$$

- Domain (inputs):
 - state $q \in Q$ (doesn't have to be start state)
 - string $w = w_1 w_2 \cdots w_n$ where $w_i \in \Sigma$

- Range (output):

states

$$qs \subseteq Q$$

Result is set of states

$\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$ is the transition function

NFA Multi-Step Transition Function

$$\hat{\delta}: Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$$

- Domain (inputs):
 - state $q \in Q$ (doesn't have to be start state)
 - string $w = w_1 w_2 \cdots w_n$ where $w_i \in \Sigma$
- Range (output):
 - states $qs \subseteq Q$

Result is set of states

(Defined recursively)

Base case

$$\hat{\delta}(q, \epsilon) = \{q\}$$

Recursively Defined Input
needs
Recursive Function

Base case

A **String** is either:

- the **empty string** (ϵ), or
- xa (non-empty string) where
 - x is a **string**
 - a is a "char" in Σ

$\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$ is the transition function

NFA Multi-Step Transition Function

$$\hat{\delta}: Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$$

- Domain (inputs):
 - state $q \in Q$ (doesn't have to be start state)
 - string $w = w_1 w_2 \cdots w_n$ where $w_i \in \Sigma$
- Range (output):
 - states $qs \subseteq Q$

(Defined recursively)

Base case $\hat{\delta}(q, \epsilon) = \{q\}$

Recursive Case

$$\hat{\delta}(q, w'w_n) =$$

where $w' = w_1 \cdots w_{n-1}$

Recursive case

Recursively Defined Input
needs
Recursive Function

A **String** is either:

- the **empty string** (ϵ), or
- xa (non-empty string) where
 - x is a **string**
 - a is a "char" in Σ

Recursive part

Recursion on recursive part

"second to last"
set of states

$$\hat{\delta}(q, w') = \{q_1, \dots, q_k\}$$

$\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$ is the transition function

NFA Multi-Step Transition Function

$$\hat{\delta}: Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$$

- Domain (inputs):
 - state $q \in Q$ (doesn't have to be start state)
 - string $w = w_1 w_2 \cdots w_n$ where $w_i \in \Sigma$
- Range (output):
 - states $qs \subseteq Q$

(Defined recursively)

Base case $\hat{\delta}(q, \epsilon) = \{q\}$

Recursive Case

$$\hat{\delta}(q, w'w_n) = \bigcup_{i=1}^k \delta(q_i, w_n)$$

where $w' = w_1 \cdots w_{n-1}$

$$\hat{\delta}(q, w') = \{q_1, \dots, q_k\}$$

For each "second to last" state, take single step on last char

Last char

Recursively Defined Input needs Recursive Function

A **String** is either:

- the **empty string** (ϵ), or
- xa (non-empty string) where
 - x is a **string**
 - a is a "char" in Σ

$\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$ is the transition function

NFA Multi-Step Transition Function

$$\hat{\delta}: Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$$

- Domain (input)
 - state $q \in Q$
 - string $w = w_1 \dots w_n$
- Range (output)
 - states $qs \subseteq Q$

Given

- An NFA (~ a “Program”)
- and Input = string of chars, e.g. “1101”

A DFA computation (~ “Program run”):

- Start in *start state*

- Repeat:

- Read 1 char from Input, and

according to *transition rules*

For each “current” state,
go to next states

... then combine all sets of “next states”

Recursive Case

$$\hat{\delta}(q, w'w_n) = \bigcup_{i=1}^k \delta(q_i, w_n)$$

where $w' = w_1 \dots w_{n-1}$

$$\hat{\delta}(q, w') = \{q_1, \dots, q_k\}$$

Still ignoring ϵ transitions!

Recursively Defined Input
needs

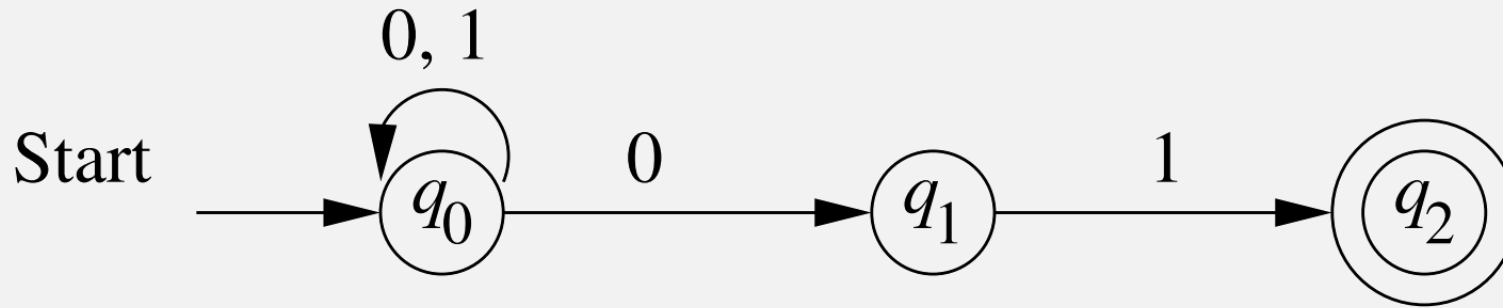
- the **empty string** (ϵ), or
- xa (non-empty string)
where
 - x is a **string**
 - a is a “char” in Σ

Base case: $\hat{\delta}(q, \epsilon) = \{q\}$

Recursive case: $\hat{\delta}(q, w) = \bigcup_{i=1}^k \delta(q_i, w_n)$
where:

$\hat{\delta}(q, w_1 \cdots w_{n-1}) = \{q_1, \dots, q_k\}$

NFA Multi-Step δ Example



- $\hat{\delta}(q_0, \epsilon) = \{q_0\}$

We haven't considered
empty transitions!

- $\hat{\delta}(q_0, 0) = \delta(q_0, 0) = \{q_0, q_1\}$

- $\hat{\delta}(q_0, 00) = \delta(q_0, 0) \cup \delta(q_1, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$

Combine result of recursive call with "last step"

- $\hat{\delta}(q_0, 001) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$

Adding Empty Transitions

- Define the set $\varepsilon\text{-REACHABLE}(q)$
 - ... to be all states reachable from q via zero or more empty transitions

(Defined recursively)

- Base case: $q \in \varepsilon\text{-REACHABLE}(q)$

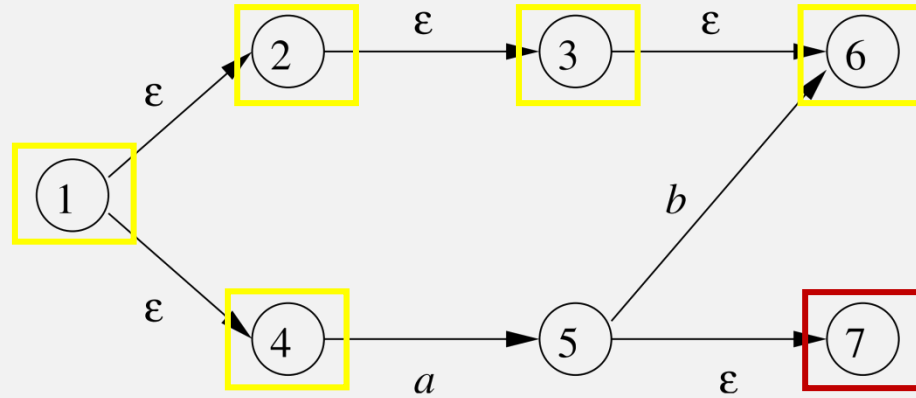
- Recursive case:

A state is in the reachable set if ...

$$\varepsilon\text{-REACHABLE}(q) = \{r \mid p \in \varepsilon\text{-REACHABLE}(q) \text{ and } r \in \delta(p, \varepsilon)\}$$

... there is an empty transition to it from another state in the reachable set

ϵ -REACHABLE Example



$$\epsilon\text{-REACHABLE}(1) = \{1, 2, 3, 4, 6\}$$

NFA Multi-Step Transition Function

$$\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$$

- Domain (inputs):
 - state $q \in Q$ (doesn't have to be start state)
 - string $w = w_1 w_2 \cdots w_n$ where $w_i \in \Sigma$
- Range (output):
 - states $qs \subseteq Q$

where $w' = w_1 \cdots w_{n-1}$

$$\hat{\delta}(q, w') = \{q_1, \dots, q_k\}$$

(Defined recursively)

Base case

$$\hat{\delta}(q, \epsilon) = \epsilon\text{-REACHABLE}(q)$$

Recursive Case

$$\hat{\delta}(q, w'w_n) =$$

$$\bigcup_{i=1}^k \delta(q_i, w_n) = \{r_1, \dots, r_\ell\}$$

NFA Multi-Step Transition Function

$$\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$$

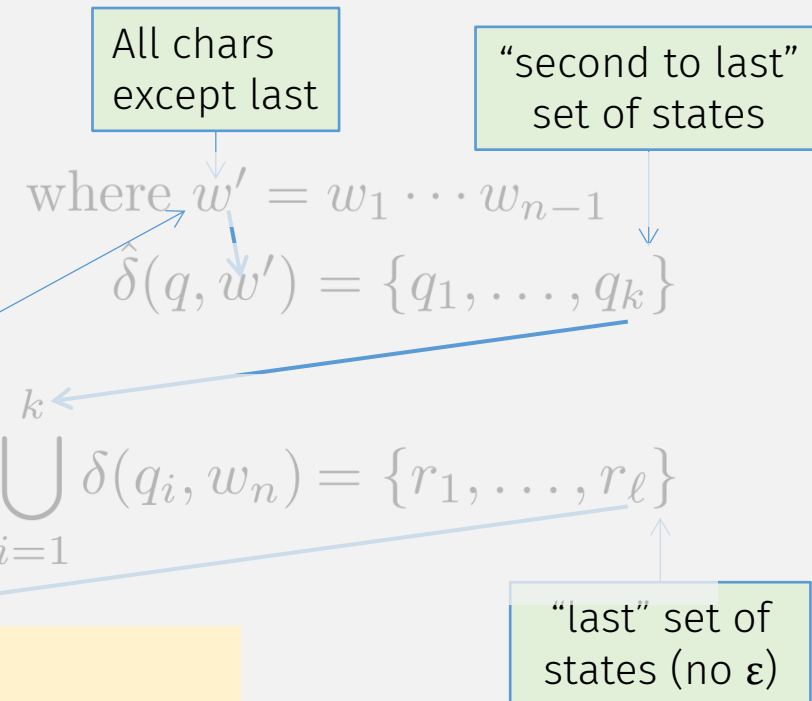
- Domain (inputs):
 - state $q \in Q$ (doesn't have to be start state)
 - string $w = w_1 w_2 \cdots w_n$ where $w_i \in \Sigma$
- Range (output):
 - states $qs \subseteq Q$

(Defined recursively)

Base case $\hat{\delta}(q, \epsilon) = \epsilon\text{-REACHABLE}(q)$

Recursive Case

$$\hat{\delta}(q, w'w_n) = \bigcup_{j=1}^{\ell} \epsilon\text{-REACHABLE}(r_j)$$



Summary: NFA vs DFA Computation

DFAs

- Can only be in one state
- Transition:
 - Must read 1 char
- Acceptance:
 - If final state is accept state

NFAs

- Can be in multiple states
- Transition
 - Has empty transitions
- Acceptance:
 - If one of final states is accept state